GRADE 11 · TERM 2

Files, Helpers & Life Online

A pencil-and-paper workbook: save data in files, build your own helpers, understand life online, then show what you know.

Name: _____

Class: _____ Date: _____

How to use this workbook

1. Work with a **pencil**. Mistakes are how we learn: rub out and try again.
2. Start each unit with **Think First**. Try it on your own before you read the explanation.
3. Read **Learn It**, then do the activities. Trace tables and code are written **by hand**.
4. At the end of each unit, tick the "I can" boxes you are sure of. Write down anything you still want to ask.
5. The **checkpoint test** is done under test conditions, with no notes.

In this workbook

1. Remembering things
2. From file to arrays and reports
3. Helpers with names
4. Helpers that answer
5. Communication, mobile technology and security
6. What is a database?
7. Term checkpoint test

> **Matches the Annual Teaching Plan:** Term 2 — text files (physical and logical names, AssignFile, Reset, Rewrite, Append, ReadLn, WriteLn, EOF, CloseFile, FileExists, try..except), populating arrays from a file and generating reports; procedures and functions with value parameters and input validation; e-communication, mobile technology, shaping and throttling, protocols, data security, blockchain, digitalisation and the gig economy; DBMS, database types and careers.

UNIT 1 Remembering things

Variables forget everything the moment a program closes. A text file remembers. You will learn the few commands that connect to a file, read it, write to it and close it, and how to stop your program crashing when the file is missing.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain why a program needs a text file
- ☐ explain the physical and the logical file name
- ☐ choose between Reset, Rewrite and Append
- ☐ trace a loop that reads a file with **while not EOF**
- ☐ write code that reads, writes and appends
- ☐ protect my program with FileExists and try..except

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

1.1 The ruler game [3]

Work with a partner. Your partner writes five names on five lines of a page: *Zinhle, Thabo, Aisha, Pieter, Naledi*. Lay a ruler **just above line 1**. To "read a line", say aloud the line **directly below the ruler**, write it on your own page, then slide the ruler down by one line. The ruler has gone past the last line when there is nothing left to read. Play it through.

(a) How many lines were you able to read? (b) What is wrong with asking for a sixth line? (c) Where must the ruler be if you want to read the whole page again?

LEARN IT

Why files? And two names for one file

A variable lives in **RAM**, which is wiped when the program closes or the power goes off in load shedding. A **text file** lives on the disk, so the data is still there tomorrow. It holds plain characters in **lines**, just like Notepad. Even a number in a text file is only text, so convert it with **StrToInt** before you calculate with it.

Picture your school locker. The number painted on the locker door is its real address: the **physical file name**, such as **Sales.txt** on the disk. The key tag in your bag is how *you* talk about it: the **logical file name**, a variable of type **TextFile**. **AssignFile** attaches the tag to the locker, so it must come first.

Delphi

```
var
  tFile : TextFile;                 // logical name: a variable in the program
begin
  AssignFile(tFile, 'Sales.txt');    // link it to the physical name on the disk
  // from here on the commands use tFile, never the physical name
```

LEARN IT

The file commands

Every file job follows the same pattern: **link, open, work, close**.

Command	What it does
AssignFile(tFile, 'Data.txt')	Links the logical name to the physical name. Always first.
Reset(tFile)	Opens an *existing* file for **reading**, at line 1.
Rewrite(tFile)	Creates a **new, empty** file for writing. If the file exists, **everything in it is erased**.
Append(tFile)	Opens an *existing* file for writing at the **end**. Old lines are kept.
ReadLn(tFile, sLine)	Reads one line into sLine and moves on to the next line.
WriteLn(tFile, sText)	Writes sText to the file as one new line.
EOF(tFile)	True when there is nothing left to read.
CloseFile(tFile)	Saves and closes the file. Never forget it.
FileExists('Data.txt')	True if that file is on the disk. Takes the physical name.

In this course every file job is written **from first principles** with these commands. There is no one-line shortcut for reading or saving a whole file.

LEARN IT

Reading, writing and appending

while not EOF(tFile) means “keep going while there is still something left to read”. Each ReadLn takes one line and moves the position on, so the loop reaches the end by itself. To save data use **Rewrite** for a fresh file, or **Append** to add to the end of an existing one. The very first time a program runs there is no file yet, and Append would fail, so it asks FileExists first.

Delphi

```
// READ every line
AssignFile(tFile, 'Names.txt');
Reset(tFile);
while not EOF(tFile) do
begin
  ReadLn(tFile, sLine);        // one line per pass
  memOut.Lines.Add(sLine);
end;
CloseFile(tFile);

// ADD a line, or create the file the first time
AssignFile(tFile, 'Sales.txt');
if FileExists('Sales.txt') then
  Append(tFile)
else
  Rewrite(tFile);
WriteLn(tFile, edtItem.Text + ',' + edtPrice.Text);
CloseFile(tFile);
```

LEARN IT

When the file is missing

Reset on a file that does not exist makes the program crash. There are two ways to be safe. **FileExists** asks *before* opening. **try..except** runs the risky code in the **try** part; if an error happens, Delphi jumps straight to the **except** part, skips the rest of the try part, and the program carries on. File errors are of the type **EInOutError**. Many programmers use both.

Delphi

```
// Method 1: ask first
if FileExists('Names.txt') then
  ...
else
  ShowMessage('Names.txt was not found');

// Method 2: try..except
AssignFile(tFile, 'Names.txt');
try
  Reset(tFile);                 // error here if the file is missing
  ...                           // read the lines
  CloseFile(tFile);
except
  on E: EInOutError do
    ShowMessage('Could not read Names.txt');
end;
```

1.2 Reset, Rewrite or Append? [5]

Circle the command that opens the file correctly for the job.

1	Show every learner name from Names.txt in a memo.	**Reset**	**Rewrite**	**Append**
2	At the start of each term, start a fresh Attendance.txt and throw away the old one.	**Reset**	**Rewrite**	**Append**
3	Add a new sale to the bottom of Sales.txt without losing the earlier sales.	**Reset**	**Rewrite**	**Append**
4	Save the very first high score, when the file does not exist yet.	**Reset**	**Rewrite**	**Append**
5	Add a late-comer to the end of Latecomers.txt.	**Reset**	**Rewrite**	**Append**

1.3 Where does the program jump? [3]

Missing.txt does **not** exist. Write exactly what **memOut** shows after this code has run.

```delphi
AssignFile(tFile, 'Missing.txt');
memOut.Lines.Add('Start');
try
  Reset(tFile);
  memOut.Lines.Add('Opened');
  CloseFile(tFile);
except
  memOut.Lines.Add('Problem');
end;
memOut.Lines.Add('Finish');
```

1.4 Be the computer: read and remember [6]

Queue.txt holds four lines: **Zinhle**, **Thabo**, **Aisha** and **Pieter**. Complete a row for every check of the loop; the values are those **after** the body has run. The last row is the check that stops the loop, so the values stay the same.

```delphi
sInitials := '';
iCount := 0;
AssignFile(tFile, 'Queue.txt');
Reset(tFile);
while not EOF(tFile) do
begin
  ReadLn(tFile, sLine);
  Inc(iCount);
  sInitials := sInitials + Copy(sLine, 1, 1);
end;
CloseFile(tFile);
```

Check	EOF(tFile)	sLine	iCount	sInitials
1				
2				
3				
4				
5				

1.5 Debug: the endless reader [3]

This code should add up the numbers in Stock.txt (one whole number per line) and show the total. It never stops and may crash. Find **three** errors and write the corrections.

Delphi

```
AssignFile(tFile, 'Stock.txt');
Reset(tFile);
iTotal := 0;
while not EOF(tFile) do
begin
  iTotal := iTotal + StrToInt(sLine);
end;
lblTotal.Caption := IntToStr(iTotal);
```

1.6 Write it: list the names [8]

Names.txt holds one learner name per line. Write code that shows every name in **memOut** with its number in front (*1. Zinhle*) and shows *5 names* (the real count) in **lblCount**. If the file does not exist, show the message *Names.txt was not found*. Variables tFile, sLine and iCount are declared.

1.7 Write it: record a sale [6]

The item is typed in **edtItem** and its price in **edtPrice**. Write code that adds one line such as *Pie,12* to the **end** of **Sales.txt**, keeping all earlier sales. If Sales.txt does not exist yet, it must be created.

UNIT 2 From file to arrays and reports

Real programs read a whole file into arrays, work with the data, and write a neat report back out. In this unit you practise the full journey: read, split, store, process, report.

By the end of this unit I can: (tick each box when you are sure)

- ☐ read the lines of a file into parallel arrays
- ☐ split a line at its commas with Pos, Copy and Delete
- ☐ keep count of how many lines were read
- ☐ total and count values using the arrays
- ☐ write a text report to a new file

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

2.1 Cut along the commas

The tuck shop keeps its stock in **Stock.txt**. Each line is one slip: *item, price in rand, quantity*.

Stock.txt

```
Bread,18,12
Milk,24,8
Eggs,55,5
Sugar,32,10
Rice,45,4
```

Imagine you cut every slip at its commas and sort the pieces into three piles: item names, prices and quantities. Place the pieces in the table below. Then answer: the third price pile holds 55. How do you find the item name and the quantity that go with it?

2.2 Sort the slips into piles [5]

Write each piece from Stock.txt in the correct pile.

Slip number	Item pile	Price pile (R)	Quantity pile
1			
2			
3			
4			
5			

LEARN IT

From a line of text to parallel arrays

Each line of the file is one **record** and the commas separate its **fields**. We keep each field in its own array. The arrays are **parallel**: position 3 of every array belongs to the same item. We do not know how many lines the file has, so the arrays are declared with a constant **MAX** and a counter **iCount** remembers how many places were really filled.
Pos(',', s) gives the position of the first comma. **Copy(s, start, length)** takes a piece. **Delete(s, start, length)** cuts a piece away, so after taking a field we delete it and its comma.

Delphi

```
const
  MAX = 50;
var
  arrItem  : array[1..MAX] of String;
  arrPrice : array[1..MAX] of Integer;
  arrQty   : array[1..MAX] of Integer;
  iCount   : Integer;                  // how many items were really read

// ----- in the event handler
iCount := 0;
AssignFile(tFile, 'Stock.txt');
Reset(tFile);
while (not EOF(tFile)) and (iCount < MAX) do
begin
  ReadLn(tFile, sLine);                // e.g. 'Sugar,32,10'
  Inc(iCount);                         // the next free place
  iPos := Pos(',', sLine);
  arrItem[iCount] := Copy(sLine, 1, iPos - 1);
  Delete(sLine, 1, iPos);              // '32,10' is left
  iPos := Pos(',', sLine);
  arrPrice[iCount] := StrToInt(Copy(sLine, 1, iPos - 1));
  Delete(sLine, 1, iPos);              // '10' is left
  arrQty[iCount] := StrToInt(sLine);
end;
CloseFile(tFile);
```

LEARN IT

Using the arrays, then writing a report

After reading, every loop runs from **1 to iCount**, never to MAX. The usual patterns work as before: a running total, a counter, and *highest so far* where we remember the **index** of the winner, because that same index also gives the winner's other fields. A report has three parts: a **heading** written once, a **body** written inside a loop, and a **summary** written once. Use **Rewrite** so that each report starts fresh.

Delphi

```
AssignFile(tOut, 'Report.txt');
Rewrite(tOut);
WriteLn(tOut, 'TUCK SHOP STOCK');              // heading: once
for i := 1 to iCount do                        // body: repeats
  WriteLn(tOut, arrItem[i] + ': R' + IntToStr(arrPrice[i]));
WriteLn(tOut, 'Items: ' + IntToStr(iCount));   // summary: once
CloseFile(tOut);
```

2.3 Be the computer: split one line [6]

Trace the seven numbered statements for the line **Sugar,32,10**. After each statement write the value of **iPos**, what **sLine** holds, and the value stored in an array (write a dash if nothing is stored).

Delphi

```
sLine := 'Sugar,32,10';
iPos := Pos(',', sLine);                              // 1
arrItem[4] := Copy(sLine, 1, iPos - 1);               // 2
Delete(sLine, 1, iPos);                               // 3
iPos := Pos(',', sLine);                              // 4
arrPrice[4] := StrToInt(Copy(sLine, 1, iPos - 1));    // 5
Delete(sLine, 1, iPos);                               // 6
arrQty[4] := StrToInt(sLine);                         // 7
```

Statement	iPos	sLine afterwards	Value stored
1			
2			
3			
4			
5			
6			
7			

2.4 Debug: reading into arrays [4]

Menu.txt has lines such as *Pie,12*. This code should read it into arrItem (String) and arrPrice (Integer). Find **four** errors.

Delphi

```
iCount := 0;
AssignFile(tFile, 'Menu.txt');
Reset(tFile);
while not EOF(tFile) do
begin
  ReadLn(tFile, sLine);
  iPos := Pos(',', sLine);
  arrItem[iCount] := Copy(sLine, 1, iPos);
  Delete(sLine, 1, iPos);
  arrPrice[iCount] := sLine;
  Inc(iCount);
end;
```

2.5 Write it: routes and fares [7]

Fares.txt has one route on each line in the form *Route,Fare,Seats* (for example *Bellville,22,15*). The arrays **arrRoute** (String), **arrFare** and **arrSeats** (Integer) and the variables below are declared. Write the statements that replace the comment, so that every line is split into the three arrays.

Delphi

```
iCount := 0;
AssignFile(tFile, 'Fares.txt');
Reset(tFile);
while (not EOF(tFile)) and (iCount < MAX) do
begin
  ReadLn(tFile, sLine);
  // your statements go here
end;
CloseFile(tFile);
```

2.6 Write it: a stock report [9]

The arrays arrItem, arrPrice and arrQty and the counter iCount are already filled. Write code that creates **StockReport.txt** in this form. The value of an item is its price times its quantity; the last line holds the total of all the values.

StockReport.txt (first, second and last line)

```
TUCK SHOP STOCK
Bread: R18 x 12 = R216
...
Total stock value: R1183
```

UNIT 3 Helpers with names

As programs grow, one long block of code becomes hard to read and to fix. A **procedure** is a named helper: write the steps once, then call its name whenever you need them. You will also learn how to hand a helper some information, and who is allowed to see which variable.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain why programs are split into procedures
- ☐ write and call a procedure without parameters
- ☐ write and call a procedure with value parameters
- ☐ explain parameter, argument and value parameter
- ☐ trace calls to find what is displayed
- ☐ tell local variables from global variables

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

3.1 Name that routine

The prefects read out the morning programme: *Stand up. Face the flag. Sing the anthem. Sit down. Read the notices. Stand up. Face the flag. Say the school pledge. Sit down. Welcome the visitors. Stand up. Face the flag. Say the prayer. Sit down. Go to class.*

(a) Find the group of steps that repeats. (b) Give the group a short name. (c) Rewrite the programme as short as you can using that name. (d) One part of the group is different each time. How could you tell the helper which one to use?

THINK FIRST · no computer needed

3.2 The photocopy

Work with a partner. Write **iMark = 50** on a card: this is your variable. Your partner is the helper. Copy the 50 onto a scrap of paper and hand **the scrap** to the helper. The helper adds 10 to the number on the scrap and writes *Inside: 60* on the board.

(a) What does your card show now? (b) Did the helper change your card? (c) What would have been different if you had handed over your original card?

LEARN IT

Procedures: write once, call by name

A **procedure** does a job but does not hand an answer back. Write it once, give it a name, and **call** it by writing its name as a statement. This avoids repeating code, makes the program easier to read and means a mistake is fixed in one place only. In a form, a procedure is declared in the **private** section of the class and written in the implementation section with **TForm1.** in front of its name, just like an event handler.

Delphi

```
type
  TForm1 = class(TForm)
    ...
  private
    procedure ShowBanner;                    // declaration
  ...
implementation
procedure TForm1.ShowBanner;                // definition
begin
  memOut.Lines.Add('*****************');
  memOut.Lines.Add('  KASI TUCK SHOP');
  memOut.Lines.Add('*****************');
end;

procedure TForm1.btnStartClick(Sender: TObject);
begin
  ShowBanner;                               // call
  memOut.Lines.Add('Open 7:00 to 16:00');
  ShowBanner;                               // call again
end;
```

LEARN IT

Value parameters

A **parameter** is a placeholder in the procedure's header that receives information. An **argument** is the actual value you give when you call. The number, order and types of the arguments must match the parameters. A **value parameter** receives a **copy**: the procedure may change its copy, but the caller's variable stays as it was.

Delphi

```
procedure TForm1.ShowLine(sItem: String; iPrice: Integer);   // 2 parameters
begin
  memOut.Lines.Add(sItem + ': R' + IntToStr(iPrice));
end;

// calls: the values in the brackets are the arguments
ShowLine('Bread', 18);
ShowLine(arrItem[3], arrPrice[3]);        // arguments can be expressions
```

LEARN IT

Local and global variables

A variable declared inside a procedure (and the procedure's own parameters) is **local**: it exists only while that procedure runs and no other code can see it. A variable declared at the top of the unit, below *Form1: TForm1*, is **global**: every procedure can read and change it and it keeps its value while the program runs. Prefer local variables and parameters; use global variables only for values that really are shared, such as the arrays and iCount that hold the data from a file. Set a global variable to a value yourself before you use it. (Think of a reminder in your own diary, which only you can read, and a notice on the staffroom board, which everybody can read and change.)

Delphi

```
var
  Form1  : TForm1;
  iTotal : Integer;                // global: all procedures can use it

procedure TForm1.AddToTotal(iAmount: Integer);
var
  iDouble : Integer;               // local: only exists in here
begin
  iDouble := iAmount * 2;
  iTotal := iTotal + iDouble;      // changes the global
end;
```

3.3 Debug: the average helper [3]

This code should show the average of 240 and 8 in **lblAvg** using a procedure. Find **three** errors.

Delphi

```
procedure TForm1.ShowAverage(iTotal, iCount);
var
  rAvg : Real;
begin
  rAvg := iTotal / iCount;
  lblAvg.Caption := FloatToStrF(rAvg, ffFixed, 8, 1);
end;

// in the button (iTotal holds 240, iCount holds 8)
ShowAverage(iTotal);
lblNote.Caption := FloatToStr(rAvg);
```

3.4 Be the computer: calls and scope [7]

iTotal is global and starts at 0. Each call adds the price plus a 10% service fee (whole rands only) to the total. Complete a row for each call. The last row is for the final statement.

Delphi

```
procedure TForm1.Sell(sItem: String; iPrice: Integer);
var
  iFee : Integer;
begin
  iFee := iPrice div 10;
  iTotal := iTotal + iPrice + iFee;
  memOut.Lines.Add(sItem + ' R' + IntToStr(iPrice + iFee));
end;

// in the button
iTotal := 0;
Sell('Airtime', 50);
Sell('Chips', 12);
Sell('Pie', 25);
memOut.Lines.Add('Total R' + IntToStr(iTotal));
```

Call	iPrice	iFee (local)	Line added to memOut	iTotal after
Sell('Airtime', 50)				
Sell('Chips', 12)				
Sell('Pie', 25)				
Last statement				

3.5 Write it: a times-table helper [6]

Write a procedure **ShowTable** with one value parameter, **iNumber** (Integer), that displays that number's times table from 1 to 12 in **memTable** (for example *3 x 7 = 21*). Then write the button code that calls it for 7 and for 12. You do not need to write the declaration in the private section.

UNIT 4 Helpers that answer

A procedure does a job. A **function** does a job and hands a value back, like an assistant who comes back from the shelf with the price. Functions let you calculate or check something in one place and use the answer anywhere, and they are the best tool for checking that what a user typed is valid.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain how a function differs from a procedure
- ☐ write a function that returns a value with Result
- ☐ call a function inside an expression or a condition
- ☐ write functions with and without parameters
- ☐ write a Boolean function to validate input
- ☐ choose between a procedure and a function

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

4.1 The mystery machine [3]

A machine takes in a number and gives a number out. Study what it did:

Number in	1	2	5	0	10
Number out	4	7	16	1	31

(a) What will come out if you put in 8? (b) Describe the rule in words. (c) Write the rule as one line of Delphi, using **n** for the number that goes in.

LEARN IT

Functions: a job that returns a value

A **function** is written like a procedure but its header ends with a colon and the **return type**. The answer is stored in the special variable **Result**. A function has no meaning on its own: the call must be **used**, so assign it, display it or test it. A procedure call, by contrast, is a statement on its own. A function may have no parameters (then write no brackets) or several. To choose between the two kinds of method ask: **do I need an answer back?** If yes, write a function; if the method only has to *do* something, such as display, clear or write to a file, write a procedure. A function that calculates should not also display: let the caller decide what to do with the answer.

Delphi

```
function TForm1.CalcVat(rPrice: Real): Real;      // one parameter, returns a Real
begin
  Result := rPrice * 0.15;
end;

function TForm1.RollDie: Integer;                  // no parameters
begin
  Result := Random(6) + 1;
end;

// using them
rVat := CalcVat(200);                              // 30
lblVat.Caption := FloatToStrF(CalcVat(StrToFloat(edtPrice.Text)), ffFixed, 8, 2);
iRoll := RollDie;                                  // no brackets
if RollDie = 6 then
  ShowMessage('A six!');
```

4.2 Be the computer: call the function [5]

Work out what each call returns. Here is the function:

Delphi

```
function TForm1.Level(iMark: Integer): Integer;
begin
  if iMark >= 80 then Result := 7
  else if iMark >= 70 then Result := 6
  else if iMark >= 60 then Result := 5
  else if iMark >= 50 then Result := 4
  else if iMark >= 40 then Result := 3
  else if iMark >= 30 then Result := 2
  else Result := 1;
end;
```

Call	Result
Level(85)	
Level(64)	
Level(50)	
Level(49)	
Level(12)	

LEARN IT

Validation with a Boolean function

Validation checks that data is acceptable (a sensible range, the right length, the right type of characters) before the program uses it. A function that returns **Boolean** is perfect: it answers True (valid) or False (not valid) and can be used straight in an **if** or a **repeat..until**. Validation checks only that the data is reasonable, not that it is true.

Delphi

```
function TForm1.IsValidMark(iMark: Integer): Boolean;
begin
  Result := (iMark >= 0) and (iMark <= 100);       // range check
end;

function TForm1.IsValidCell(sCell: String): Boolean;
var
  i : Integer;
begin
  Result := (Length(sCell) = 10) and (Copy(sCell, 1, 1) = '0');   // length and format
  for i := 1 to Length(sCell) do
    if not (sCell[i] in ['0'..'9']) then                          // type check
      Result := False;
end;

// keep asking until the answer is valid
repeat
  sCell := InputBox('Cell number', 'Enter 10 digits starting with 0:', '');
until IsValidCell(sCell);
```

4.3 Procedure or function? [6]

Circle the better choice for each task.

1	Show a welcome banner in a memo.	**Procedure**	**Function**
2	Calculate the price of an item including VAT.	**Procedure**	**Function**
3	Check whether a cell number is valid.	**Procedure**	**Function**
4	Clear all the edit boxes on the form.	**Procedure**	**Function**
5	Work out the average of the marks in an array.	**Procedure**	**Function**
6	Display the times table of a number.	**Procedure**	**Function**

4.4 Debug: two functions [4]

Find **four** errors in this code. The program should show the average of 240 and 8 and a pass/fail decision.

Delphi

```
function TForm1.IsPass(iMark : Integer) : Boolean;
begin
  if iMark >= 50 then
    Result := True;
end;

function TForm1.CalcAverage(iTotal, iCount : Integer) : Integer;
begin
  Result := iTotal / iCount;
end;

// in the button
CalcAverage(240, 8);
lblAvg.Caption := IntToStr(rAvg);
```

4.5 Write it: a PIN check [9]

A PIN is valid only if it has **exactly 4 characters** and they are **all digits**. Write a Boolean function **IsValidPin(sPin: String)**. Then write the code for the button that keeps asking for a PIN with an InputBox until a valid one is entered.

UNIT 5 Communication, mobile technology and security

Paper 2 asks you to explain, compare and discuss. In this unit you meet the ways we communicate online, the technology in your pocket and the ideas that keep data safe. You also practise answering in **paragraphs**: a marker reads a whole paragraph, but may stop crediting a list after the first few bullets.

By the end of this unit I can: (tick each box when you are sure)

- ☐ tell the e-communication forms apart and say if each is synchronous or asynchronous
- ☐ explain GPS, Bluetooth, Wi-Fi, access points and 4G/5G
- ☐ explain shaping, throttling, SMTP, POP3 and IMAP
- ☐ explain strong passwords, encryption and multi-layer verification
- ☐ explain what a blockchain is and why it is hard to cheat
- ☐ describe digitalisation and the gig economy
- ☐ write a theory answer as a paragraph, not a list

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

5.1 Sports day is cancelled

It rains on the morning of sports day and the principal must tell 800 learners and parents by 06:30. Many parents have no data bundle, some only have old phones with no apps, and the school also has a page on social media.

(a) Name two different ways to reach everybody and give a reason for each. (b) Which of your choices works without the internet? (c) For which kind of message must both people be online at the same moment?

5.2 The secret shift [4]

Long before computers, people hid messages by moving every letter a fixed number of places along the alphabet. In the message below every letter was moved **3 places forward** (A became D, B became E and so on). Move each letter **3 places back** to read it. Then hide your own word: write TAXI with every letter moved **5 places forward**.

1	2	3	4	5	6	7	8	9	10	11	12
O	R	D	G	V	K	H	G	G	L	Q	J

5.3 The honest notebook and the cheat [6]

Four friends share a notebook that records who paid whom, one page per payment. To make cheating hard every page also shows a **fingerprint**: **the last two digits of (the amount + 3 × the fingerprint of the page before)**. Page 1 uses 0 as the fingerprint of the page before. Work out the fingerprint of every page. (Example: page 2 = last two digits of 20 + 3 × 50 = 170, so the fingerprint is 70.) Then a cheat secretly changes the amount on page 2 from R20 to **R80**: recalculate the fingerprints of all four pages with the same rule.

Page	Amount (R)	Fingerprint	Cheat: amount now (R)	Cheat: new fingerprint
1	50		50	
2	20		80	
3	75		75	
4	40		40	

5.4 What does the cheat have to do? [3]

The friends each keep a copy of the notebook. In two or three sentences, explain how the fingerprints show that page 2 was changed, and why it is so much work for a cheat to hide it.

LEARN IT

Ways to communicate online

E-communication means sending and receiving information with electronic devices and the internet.

Form	What it is	Good to know
Microblogging	Very short public posts shared with followers (for example on X)	Quick news that is easy to reshare
SMS	A short text sent over the cellular network	Needs *no internet*; banks use it for one-time PINs
Instant messaging	Real-time chat, voice notes and pictures over the internet, one-to-one or in groups (WhatsApp)	Free over data or Wi-Fi; informal
Social media	Websites and apps where people make profiles, share and interact (Facebook, Instagram, TikTok)	Huge audience; privacy and digital footprint risks

LEARN IT

Video, audio and calls

Form	What it is	Good to know
Videocasting	A video series published on a channel such as YouTube, recorded or live	Viewers choose when to watch
Podcasting	An audio series you subscribe to and download	Can be played offline
VoIP	Voice (and video) calls carried over the internet instead of the phone network	Free between users of the same app; needs a stable connection, enough bandwidth and low delay (latency)
Video conferencing	A live meeting where several people see and hear each other (Teams, Meet, Zoom)	Shows facial expressions; uses a lot of data

Communication is **synchronous** when everybody takes part at the same moment (a call or a live video meeting) and **asynchronous** when the receiver can answer later (SMS, e-mail, a podcast episode).

LEARN IT

Mobile devices and wireless technologies

A **smartphone** is a pocket computer with a touchscreen, camera, GPS and an **app store**. A **feature phone** makes calls and sends SMS with a few extras but has no full app store. Other mobile devices are **tablets** and **wearables** such as smartwatches. An **app** is a program for a mobile device, downloaded from an app store; a **browser** opens websites. Mobility is convenient, but battery life, small screens and the cost of data are limits.

Technology	What it does	Remember
GPS	A receiver works out its position from the signals of satellites	Powers navigation and e-hailing apps; it only listens, it does not send
Bluetooth	A short-range wireless link between two devices	About 10 m: earphones, speakers, a car's hands-free kit
Wi-Fi	A wireless link to a network and the internet through a router	Tens of metres: home, school, office
Access point	A device that connects Wi-Fi devices to the wired network	A school uses many so that every classroom has a signal
4G / 5G	Mobile internet through the cellular towers of a network provider	Works across the country; 5G is faster and has less delay

LEARN IT

Shaping, throttling and protocols

A **protocol** is a set of rules that devices follow so that they understand each other. Internet service providers (ISPs) also manage how fast your connection is. **Shaping** means the ISP gives some types of traffic priority over others, for example video calls over large downloads, to keep the network running smoothly. **Throttling** means the ISP deliberately slows your connection, for example after you have used up the fast data in your package.

Protocol	Its job
SMTP	*Sends* e-mail from your program to the mail server and on to the receiver's server
POP3	*Downloads* e-mail to one device, normally removing it from the server
IMAP	Lets you read e-mail while it *stays on the server*, so every device shows the same mailbox
VoIP	Carries a voice call as data across the internet
HTTPS	Fetches web pages with encryption (the padlock)

LEARN IT

Passwords, encryption and multi-layer verification

A **strong password** is long (at least 12 characters), mixes upper case, lower case, digits and symbols, is not based on your name or birth year, and is **different for every account**, so that one leak does not open everything. **Encryption** scrambles data (*plaintext*) into unreadable *ciphertext* using a method and a secret **key**. Only someone with the right key can turn it back. It protects data that is stored (an encrypted phone) and data on the move (HTTPS and WhatsApp). **Multi-layer (multi-factor) verification** asks for more than one proof of who you are, taken from different groups: something you *know* (a password or PIN), something you *have* (your phone, which receives a one-time PIN) or something you *are* (a fingerprint or face). A thief who steals only the password still cannot get in.

LEARN IT

Blockchain, digitalisation and the gig economy

A **blockchain** is a digital ledger (record book) kept as a chain of **blocks**. Each block holds a group of records and the fingerprint of the block before it, so the blocks are linked. Copies are kept on many computers, with no single owner, so nobody can quietly change one block: it would no longer match the next one, and the other copies would disagree. Its purpose is to record transactions in a way that is open, shared and very hard to cheat. Cryptocurrencies such as Bitcoin use it, and so do systems that track goods from farm to shop.

Digitalisation means moving tasks from paper and people to digital systems. It brings speed, accuracy and new jobs, but some routine jobs disappear and people without devices or data are left behind. The **gig economy** is work made up of short jobs (“gigs”) arranged through online platforms, such as e-hailing drivers, food couriers and freelancers. Workers choose their hours and can earn extra money, but have no job security, no paid leave or pension and an unpredictable income.

LEARN IT

Answer in paragraphs, not lists

A marker reads **every** sentence of a paragraph, but may stop crediting a bullet list after the first few bullets. Give each point a **statement**, an **explanation** (because…) and an **example** (for example…). Check the marks: each mark needs one separate, explained point.

Question: Explain what throttling is. [3]
Model answer: "Throttling is when an internet service provider deliberately slows down a customer's connection. This usually happens after the customer has used all the fast data in their package, so videos buffer and downloads take much longer. For example, a learner who has used up 10 GB may find that the connection crawls until the next month starts."

5.5 Match the term [10]

Write the term from the word bank next to its description.
Word bank: access point · Bluetooth · GPS · IMAP · microblogging · podcasting · POP3 · SMS · SMTP · VoIP

Description	Term
Needs no internet; sent over the cellular network	
Very short public posts that followers can read and reshare	
An audio series you subscribe to and can play offline	
Voice calls carried over the internet	
Sends an e-mail from your program to the mail server	
Downloads e-mail to one device, normally removing it from the server	
Keeps e-mail on the server so that your phone and laptop show the same mailbox	
Works out a position from the signals of satellites	
A short-range wireless link between a phone and earphones	
Connects Wi-Fi devices to the wired network	

5.6 Shaping or throttling? [4]

Zinhle's provider lets her video calls run smoothly even when her brother downloads large games, but after she has used 20 GB it slows her whole line. Name the practice in each case and explain the difference in a short paragraph.

5.7 Encryption in a paragraph [4]

Explain what encryption is, and how it protects the photos on a phone that has been stolen.

5.8 Passwords and multi-layer verification [5]

Thandi uses the password *thandi2008* for her e-mail, her banking app and her social media. In a paragraph, explain why this is risky and how multi-layer (multi-factor) verification would make her banking app safer.

5.9 Blockchain in a paragraph [4]

Explain what a blockchain is, and why it is very hard to change an old record on it.

5.10 Digitalisation and the gig economy [4]

Mr Dube drives for an e-hailing app and has no other job. Discuss **two advantages** and **two disadvantages** of this kind of work for him. Write full sentences.

UNIT 6 What is a database?

Next term you will use databases in Delphi. This unit is the paper-based preparation: what a database and a DBMS are, why a DBMS beats a paper register or a text file, the different kinds of database, and the people who work with them.

By the end of this unit I can: (tick each box when you are sure)

- ☐ define a database and a DBMS
- ☐ name the functions of a DBMS and give examples of DBMS software
- ☐ name the parts of a table: field, record and primary key
- ☐ tell desktop, server and distributed databases apart
- ☐ name three database careers and say what each person does
- ☐ explain why a DBMS is better than a paper register or a text file

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

6.1 The paper register

This is a page of the school's paper register.

LearnerNo	Surname	Name	Grade	Cell
1001	Mokoena	Thabo	11	082 555 0101
1002	Naidoo	Priya	10	083 555 0102
1003	Botha	Jan	11	084 555 0103
1004	Dlamini	Zinhle	12	072 555 0104
1005	Khumalo	Sipho	11	073 555 0105
1006	Pillay	Aisha	10	074 555 0106

(a) List the Grade 11 learners. (b) Find the cell number of learner 1004. (c) The real register has 1 200 learners. Would (b) still be quick, and what would make it quicker? (d) The principal and the office each keep their own copy. Zinhle gets a new cell number: the office updates its copy but the principal's copy is not changed. What goes wrong?

LEARN IT

Database and DBMS

A **database** is an organised collection of related data, stored in **tables** so that it can be found, changed and shared quickly. In a table each **row** is a **record** (everything about one learner or product) and each **column** is a **field** (one kind of information). The **primary key** is the field that is different for every record and so identifies it, such as LearnerNo.

A **DBMS** (database management system) is the **software** that creates a database and manages it. Its functions are to: **store and organise** the data; let users **retrieve** it by searching, sorting and filtering (*querying*); **add, change and delete** records while **validating** them; control who may do what (**security**); make **backups** and recover after a crash; let **many users** work at once; and produce **reports**. Examples are Microsoft Access, MySQL, Microsoft SQL Server, Oracle and PostgreSQL. Delphi is not a DBMS: a Delphi program *connects* to one.

LEARN IT

Types of database and database careers

A **desktop (personal) database** is stored on one computer and used by one person or a few, such as an Access database for a small shop. A **server (centralised) database** is stored on a central server and many users reach it over a network, such as a school's admin system used by the teachers and the office. A **distributed database** is spread over several computers or servers in different places that work together and keep one another up to date, such as a bank's records or a national learner database used in every province.

A **database administrator (DBA)** looks after the database: security, backups, user access and keeping it fast. A **database programmer (developer)** writes the queries and code that let applications use the data. A **database analyst** studies what the organisation needs and designs the tables and how they are linked.

6.2 Desktop, server or distributed? [4]

Circle the type of database that fits best.

1	A teacher keeps her marks in an Access database on her own laptop.	**Desktop** **Server** **Distributed**
2	All the teachers and the school office use one admin system that is stored on the school's server.	**Desktop** **Server** **Distributed**
3	A bank keeps copies of its customer records in Johannesburg, Durban and Cape Town, and the copies keep one another up to date.	**Desktop** **Server** **Distributed**
4	The spaza shop owner keeps a small stock database on the shop's only computer.	**Desktop** **Server** **Distributed**

6.3 Match the term [8]

Write the term from the word bank next to the situation.
Word bank: backup · database administrator · database analyst · database programmer · multi-user access · querying · security · validation

Situation	Term
A learner's mark of 150 is rejected.	
Only the office staff can change fees; teachers may only view them.	
The disk crashes at night, but the data is restored the next morning.	
Ten teachers capture marks at the same time without overwriting one another.	
The principal asks for all Grade 11 learners who owe fees and the answer appears in seconds.	
Designs the tables and how they are linked.	
Makes sure backups are made and only authorised users can log in.	
Writes the queries and code that let an app read and update the data.	

6.4 A DBMS or a text file? [4]

This term you stored data in text files. Think about searching, checking the data, several users and security. Explain, in a paragraph, why a school should use a DBMS rather than text files to keep the records of 1 200 learners.

6.5 Design a table [6]

Design the table **tblProduct** for the tuck shop. On the grid draw a table with **five field names as headings**, circle the **primary key** and fill in **three records** of sample data.

TERM CHECKPOINT

Files, methods and communication

Time: 90 minutes **Total:** 64 marks **Mark:** _____ / 64

No notes, no computer. Show all your working. Write Delphi code neatly and use correct indentation.

SECTION A — Trace and predict [10]

A1 [6]

Quiz.txt holds four lines:

Quiz.txt

```
Thabo,14
Aisha,9
Lwazi,17
Mpho,12
```

Complete a row for every check of the loop. The last row is the check that ends the loop.

```Delphi
iCount := 0;
iPass := 0;
AssignFile(tFile, 'Quiz.txt');
Reset(tFile);
while not EOF(tFile) do
begin
  ReadLn(tFile, sLine);
  Inc(iCount);
  iPos := Pos(',', sLine);
  arrName[iCount] := Copy(sLine, 1, iPos - 1);
  arrMark[iCount] := StrToInt(Copy(sLine, iPos + 1, Length(sLine)));
  if arrMark[iCount] >= 12 then
    Inc(iPass);
end;
CloseFile(tFile);
```

Check	EOF(tFile)	iCount	arrMark[iCount]	iPass
1				
2				
3				
4				
5				

A2 **[4]**

iGlobal is a global variable. Write exactly what **memOut** shows.

Delphi

```
procedure TForm1.Boost(iScore: Integer);
begin
  iScore := iScore * 2;
  iGlobal := iGlobal + iScore;
end;

// in the button
iGlobal := 0;
iMark := 10;
Boost(iMark);
Boost(iMark + 3);
memOut.Lines.Add('iMark = ' + IntToStr(iMark));
memOut.Lines.Add('iGlobal = ' + IntToStr(iGlobal));
```

SECTION B — Find the errors [6]

B1 **[3]**

This code should add the name typed in **edtName** as a new line at the **end** of the existing file **Visitors.txt**, keeping the old lines. Find and correct **three** errors.

Delphi

```
AssignFile(tFile, 'Visitors.txt');
Rewrite(tFile);
WriteLn(edtName.Text);
CloseFile(tFile)
ShowMessage('Saved');
```

B2 **[3]**

This function should return True when an age is from 14 to 19 (inclusive), and the button should use it. Find and correct **three** errors.

Delphi

```
function TForm1.IsValidAge(iAge : Integer) : Boolean;
begin
  if (iAge >= 14) or (iAge <= 19) then
    Result := True;
end;

procedure TForm1.btnCheckClick(Sender: TObject);
var
  iAge : Integer;
begin
  iAge := StrToInt(edtAge.Text);
  if IsValidAge then
    ShowMessage('Age accepted')
  else
    ShowMessage('Age must be 14 to 19');
end;
```

SECTION C — Theory [18]

C1 **[5]**

Choose the word from the word bank for each description.
Word bank: access point · Bluetooth · GPS · IMAP · SMTP

Description	Answer
The protocol that sends e-mail	
Keeps e-mail on the server so that every device shows the same mailbox	
A short-range link between a phone and earphones	
Works out a position from satellite signals	
Connects Wi-Fi devices to the wired network	

C2 **[3]**

Differentiate between **shaping** and **throttling**.

C3 **[4]**

Explain how **encryption** and **multi-layer verification** each help to protect an online bank account. Write in sentences.

C4 **[3]**

Explain what a **blockchain** is and why it is difficult to change an old record on it.

C5 **[3]**

Define a **DBMS** and describe **two** of its functions.

SECTION D — Write the code [30]

D1 [9]

Results.txt has one learner on each line in the form *Name,Mark* (for example *Nomsa,72*). The constant **MAX = 50** and the global arrays **arrName** (String), **arrMark** (Integer) and the global counter **iCount** are declared. (In all the code questions MAX, arrName, arrMark and iCount are declared as global; declare any other variables that you use.) Write a procedure **LoadResults** with **no parameters** that reads the file into the arrays and sets iCount. If the file does not exist it must show *Results.txt not found*.

D2 [4]

Write a function **CalcAverage** with no parameters that returns, as a Real, the average of the marks in arrMark (positions 1 to iCount). If iCount is 0 it must return 0. (For the marks 72, 45, 88, 50, 63 the answer is 63.6.)

D3 **[6]**

Write a procedure **ShowAbove** with one value parameter, **iLimit** (Integer), that displays in **memOut** the name and mark of every learner whose mark is **greater than iLimit**, in the form *Sizwe: 88*. Then write the button line that calls it with the limit 60.

D4 **[6]**

The arrays and iCount are filled. Write code that creates **ResultsReport.txt**. The first line is the heading, then one line for each learner in the form *Nomsa: 72 PASS* (50 or more is a PASS, otherwise FAIL), and a last line that shows how many learners passed, such as *Passed: 4*.

ResultsReport.txt

```
RESULTS REPORT
Nomsa: 72 PASS
Pieter: 45 FAIL
...
Passed: 4
```

D5 **[5]**

A password is valid only if it has **at least 8 characters** and contains **at least one digit**. Write a Boolean function **IsValidPassword** with one String parameter **sPass**.

COMING UP NEXT TERM · warm-up

Registers and library cards

In Term 3 you will use Delphi to work with **tables** that live in a database. Before you touch a computer, think about two paper systems.

1. The class register. Each learner has a row. The columns are LearnerNo, Surname, Name and Cell. (a) What is one row called? What is one column called? (b) Which column must never be the same for two learners?

2. The library card box. Each time a learner borrows a book, the librarian writes a card with the learner's name, cell number, the book title and the date due. Zinhle borrows three books this month. (c) Which pieces of information are written down three times? (d) What could go wrong if Zinhle's cell number changes? (e) How could the librarian store the cell number only **once** and still know which books Zinhle has?